

JSON формат

- Введение в основы JSON
- Кодирование данных
- Декодирование данных
- Примеры преобразований
- Обработка ошибок
- Итоги

Введение в основы JSON

Что такое JSON

JSON (JavaScript Object Notation, представление объектов JavaScript) - это независимый от платформы и языка способ сериализации **объектов в открытый текст**.

JSON - простой, способ хранить и передавать структурированные данные. С помощью простого синтаксиса можно хранить все, что угодно, начиная от одного числа до строк, массивов и объектов. После создания строки JSON, ее легко отправить другому приложению или в другое место сети, так как она представляет собой **простой текст**.

JSON имеет следующие преимущества:

- компактен;
- JSON-представления легко читаются и составляются как человеком, так и компьютером;
- легко преобразовать в структуру данных для большинства языков программирования (числа, строки, логические переменные, массивы и так далее);
- многие языки программирования имеют функции и библиотеки для чтения и создания структур JSON.

Важно. JSON-формат — это текстовое представление массивов и объектов (как правило) используемое для простого обмена информацией между различными приложениями.

Для чего используется JSON

Наиболее распространенное использование JSON - **пересылка данных от сервера к браузеру**. Обычно данные JSON доставляются с помощью технологии **AJAX** (Asynchronous Javascript and XML), которая позволяет обмениваться данными браузеру и серверу **без необходимости перезагружать страницу**.

Пример цепочки действий, использующих формат JSON:

1. Пользователь нажимает миниатюру продукта в онлайн магазине.
2. JavaScript, выполняющийся **на браузере**, генерирует запрос **AJAX** к **серверному скрипту** PHP, передавая ID выбранного продукта.
3. Используя принятый ID, скрипт PHP извлекает название продукта, описание, цену и другую информацию из базы данных. Затем формирует из данных **строку JSON** и отправляет ее обратно браузеру.
4. JavaScript, выполняющийся на браузере, получает **строку JSON**, декодирует ее и выводит информацию о продукте на странице для пользователя.

Другой способ использовать JSON - отправка данных от браузера на сервер, передавая строку JSON в **качестве параметра** GET-запроса.

Таким образом. Изучение формата особенно важно для тех, кто имеет желание изучить клиент-серверное взаимодействие сценариев (REST API).

Как создать строку JSON

Есть несколько основных правил для создания строки JSON:

- **Строка JSON** содержит либо **массив** значений (JSON массив), либо **объект** (JSON объект).
- **Массив** заключается в квадратные скобки ([и]) и содержит разделенный запятой **список значений**.
- **Объект** заключается в фигурные скобки ({ и }) и содержит разделенный запятой список пар **имя / значение**.
- **Пара имя/значение** состоит из **имени** поля, заключенного в двойные кавычки, за которым следует **двоеточие** (:) и **значение** поля. **Значение может быть:**
 - числом (целым или с плавающей точкой);
 - строкой (в двойных кавычках);
 - логическим значением (true или false);
 - другим массивом (заключенным в квадратные скобки);
 - другим объектом (заключенным в фигурные скобки);
 - значением null.

Обратите внимание. Желание подробно объяснить материал иногда приводит к путанице терминологий. Итак, **строку JSON** часто называют **объектом JSON** (здесь я все-таки употребляю слово строка). В свою очередь **строка JSON** может содержать внутри себя массив – **JSON массив**, или объект – **JSON объект**.

Чтобы включить двойные кавычки в строку, нужно использовать обратную косую черту: \".

Так же, как и во многих языках программирования, можно помещать управляющие символы и шестнадцатеричные коды в строку, предваряя их обратной косой чертой (\n, \t и \.).

Примеры формирования строки JSON:

1. Индексный одномерный PHP массив:

```
$array = ["Hamlet", "W.Shakespeare", 1601, "Tragedy", true];
```

- **JSON-представление индексного одномерного массива (JSON массив):**

```
'["Hamlet","W.Shakespeare",1601,"Tragedy",true]';
```

2. Ассоциативный одномерный PHP массив:

```
$array = [  
    "name" => "Hamlet",  
    "author" => "W.Shakespeare",  
    "year" => 1601,  
    "genre" => "Tragedy",  
    "bestseller" => true  
];
```

- **JSON-представление ассоциативного одномерного массива (JSON объект):**

```
'{"name":"Hamlet","author":"W.Shakespeare","year":1601,"genre":"Tragedy","bestseller":true}';
```

Обратите внимание. По умолчанию ассоциативный PHP-массив имеет JSON-представление в виде строкового **литерала объекта** (об этом дальше).

PHP имеет встроенные функции для **кодирования** и **декодирования** данных JSON. Это функции **json_encode()** и **json_decode()** соответственно.

Обе функции работают только со строковыми данными в кодировке UTF-8.

Кодировка русских символов

Русские символы в строке JSON **по умолчанию** будут отображаться в кодировке UTF-8.

Примечание. Для понимания материала необходимо различать следующие термины:

- **Unicode** — набор символов;
- **UTF-8** — алгоритм кодирования кода символа в машинный язык.

Набор символов — это таблица символов с уникальными номерами. Unicode — стандарт кодирования символов, позволяющий представить знаки почти всех письменных языков. Представляемые в юникоде символы кодируются целыми числами без знака. Эти числа называются **кодами символов** в юникоде или просто Unicode.

Кодировка — это алгоритм, который преобразует **коды символов** в двоичный код, понятный машине.

В кодировке UTF-8 унаследованы однобайтные коды таблицы символов **ASCII-7**, т.е. одним байтом кодируются латинские буквы, цифры и специальные символы. Русские буквы (кириллица) представляются двухбайтными кодами в соответствии с таблицей **Unicode**.

Unicode некоторых русских букв:

- П — 041F
- р — 0440
- и — 0438
- в — 0432
- е — 0435

- т — 0442

Отсюда следует, что строка "Привет" в JSON формате будет выглядеть следующим образом (префикс \u - обязателен):

```
"\u041f\u0440\u0438\u0432\u0435\u0442"
```

Действие по кодированию русских символов можно **переопределить** с помощью специального флага.

Кодирование данных

В PHP функция **json_encode()** используется для **кодирования** значения в JSON формат.

json_encode

Функция **json_encode ()** — возвращает JSON-представление данных.

Описание

```
json_encode($value [, $flags, $depth]);
```

Возвращает строку, содержащую JSON-представление для указанного **value**. На кодирование влияет параметр **flags**.

Параметры

- **value** – значение, которое будет закодировано. Может быть любого типа, кроме resource.
- **flags** – необязательные дополнительные константы влияющие на кодировку. Констант достаточно много, мы рассмотрим **JSON_FORCE_OBJECT** и **JSON_UNESCAPED_UNICODE**.
- **depth** – необязательный, устанавливает максимальную глубину кодирования. Должен быть больше нуля.

Возвращаемые значения

Возвращает строку, закодированную JSON или **false** в случае возникновения ошибки.

example_1. JSON-кодирование индексного одномерного массива

```
<?php

    // одномерный индексный массив

    $array_auto = ['Toyota', 'Toyota Land Cruiser', 'Дизельный',
    '249'];

    echo "<h2>Одномерный индексный массив</h2>";

    echo "<pre>";

    var_dump($array_auto);

    echo "</pre>";

    // преобразуем массив в json представление

    $json_auto = json_encode($array_auto);

    echo "<h2>JSON-представление одномерного индексного
    массива</h2>";

    echo "<pre>";

    var_dump($json_auto);

    echo "</pre>";

?>
```

example_2. JSON-кодирование ассоциативного одномерного массива

```
<?php

    // одномерный ассоциативный массив

    $array_auto = ['brand'=>'Toyota', 'model'=>'Toyota Land
    Cruiser', 'engine'=>'Дизельный', 'power'=>'249'];

    echo "<h2>Одномерный ассоциативный массив</h2>";

    echo "<pre>";
```

```

var_dump($array_auto);

echo "</pre>";

// преобразуем массив в json представление

$json_auto = json_encode($array_auto);

echo "<h2>JSON-представление одномерного ассоциативного
массива</h2>";

echo "<pre>";

var_dump($json_auto);

echo "</pre>";

```

?>

Важно. Действия, выполняемые **json_encode** по умолчанию:

- **Индексный PHP массив** кодирует в **JSON массив**.
- **Ассоциативный PHP массив** кодирует в **JSON объект**.

Действие функции **json_encode** можно **переопределить** с помощью флага **JSON_FORCE_OBJECT**.

Флаг **JSON_FORCE_OBJECT** – обозначает сохранять в строке JSON **объект** вместо массива при кодировании **индексного** массива. При этом ключами объекта будут числа.

example_3. Использование флага **JSON_FORCE_OBJECT** при кодировании

```

<?php

// одномерный индексный массив

$array_auto = ['Toyota', 'Toyota Land Cruiser', 'Дизельный',
'249'];

echo "<pre>";

var_dump($array_auto);

```

```

echo "</pre>";

// преобразуем массив в json-представление

$json_auto = json_encode($array_auto);

echo "<h2>JSON-представление индексного массива в виде
массива</h2>";

echo "<pre>";

var_dump($json_auto);

echo "</pre>";

// преобразуем массив в json-представление

// используем флаг - JSON_FORCE_OBJECT

$json_auto = json_encode($array_auto, JSON_FORCE_OBJECT);

echo "<h2>JSON-представление индексного массива в виде
объекта</h2>";

echo "<pre>";

var_dump($json_auto);

echo "</pre>";

```

?>

Важно. Действия, выполняемые **json_encode**:

- **Индексный PHP массив** кодирует в **JSON массив / JSON объект** (в зависимости от флага).
- **Ассоциативный PHP массив** кодирует в **JSON объект**.

Действие по кодированию русских символов можно **переопределить** с помощью специального флага - **JSON_UNESCAPED_UNICODE**. Флаг означает - не кодировать многобайтовые символы Unicode (по умолчанию они кодируются как \uXXXX).

Например:

```
<?php

// пример использования флага JSON_UNESCAPED_UNICODE
// при кодировке русских символов

echo json_encode("Привет");

// "\u041f\u0440\u0438\u0432\u0435\u0442"

echo "<p>";

echo json_encode("Привет", JSON_UNESCAPED_UNICODE);

// "Привет"
```

Декодирование данных

Расшифровать или декодировать JSON-данные так же просто, как и закодировать. Для преобразования закодированной JSON-строки в соответствующий тип данных PHP используется функция **json_decode()**.

json_decode

Функция **json_decode ()** — декодирует строку JSON.

Описание

```
json_decode($value [, $assoc, $depth, $flags]);
```

Принимает закодированную в **JSON строку** и преобразует её в **переменную PHP**.

Параметры

- **value** – строка JSON для декодирования.
- **assoc** –
 - **true**, объекты JSON будут возвращены как ассоциативные массивы (array);
 - **false**, объекты JSON будут возвращены как объекты (object);

- **null**, объекты JSON будут возвращены как ассоциативные массивы (array) или объекты (object) в зависимости от того, установлена ли **JSON_OBJECT_AS_ARRAY** в flags.
- **depth** – необязательная максимальная глубина вложенности структуры, для которой будет производиться декодирование.
- **flags** - необязательные дополнительные константы влияющие на кодировку. Констант достаточно много, мы рассмотрим **JSON_OBJECT_AS_ARRAY**.

Возвращаемые значения

Возвращает данные JSON, преобразованные в соответствующие типы PHP. Значения **true**, **false** и **null** возвращаются как **true**, **false** и **null** соответственно.

Null также возвращается, если JSON не может быть преобразован или закодированные данные содержат вложенных уровней больше, чем указанный предел вложенности.

example_4. JSON-декодирование одномерного массива

```
<?php

// одномерный индексный массив

$array_auto = ['Toyota', 'Toyota Land Cruiser', 'Дизельный',
'249'];

// преобразуем массив в json-представление

$json_auto = json_encode($array_auto);

echo "<h2>JSON-представление массива</h2>";

echo "<pre>";

var_dump($json_auto);

echo "</pre>";

// выполним декодирование json строки обратно в массив

$new_array = json_decode($json_auto);
```

```
echo "<h2>Массив, полученный из JSON строки</h2>";

echo "<pre>";

// массив

var_dump($new_array);

echo "</pre>";

// второй элемента массива

// var_dump($new_array[1]);

?>
```

example_5. JSON-декодирование объекта

```
<?php

// одномерный ассоциативный массив

$array_auto = ['brand'=>'Toyota', 'model'=>'Toyota Land
Cruiser', 'engine'=>'Дизельный', 'power'=>'249'];

// преобразуем ассоциативный массив в json объект

$json_auto = json_encode($array_auto);

echo "<h2>JSON объект ассоциативного массива</h2>";

echo "<pre>";

var_dump($json_auto);

echo "</pre>";

// выполним декодирование json строки

$object_auto = json_decode($json_auto);

echo "<h2>PHP объект, полученный из строки JSON
объекта</h2>";

echo "<pre>";

var_dump($object_auto);

echo "</pre>";
```

```
// получение доступа к свойству объекта

// echo $object_auto->model;

?>
```

Важно. Действия, выполняемые **json_decode** по умолчанию:

- **JSON массив** декодирует в **PHP массив**.
- **JSON объект** декодирует в **PHP объект**.

Действие функции `json_decode` можно **переопределить** с помощью флага `JSON_OBJECT_AS_ARRAY`.

Флаг **JSON_OBJECT_AS_ARRAY** – декодирует JSON объект в ассоциативный массив PHP. Эта опция может быть задана автоматически, если вызвать **json_decode()** указав вторым параметром **true**.

example_6. Использование флага `JSON_OBJECT_AS_ARRAY`

```
<?php

// одномерный ассоциативный массив

$array_auto = ['brand'=>'Toyota', 'model'=>'Toyota Land
Cruiser', 'engine'=>'Дизельный', 'power'=>'249'];

// преобразуем ассоциативный массив в json объект

$json_auto = json_encode($array_auto);

echo "<h2>JSON объект ассоциативного массива</h2>";

echo "<pre>";

var_dump($json_auto);

echo "</pre>";

// выполним декодирование json строки

$new_object = json_decode($json_auto);

echo "<h2>PHP объект, полученный из строки JSON
```

```
объекта</h2>";

echo "<pre>";

var_dump($new_object);

echo "</pre>";

// выполним декодирование json строки

$new_array = json_decode($json_auto, JSON_OBJECT_AS_ARRAY);

echo "<h2>Ассоциативный PHP массив, полученный из строки JSON
объекта</h2>";

echo "<pre>";

var_dump($new_array);

echo "</pre>";

?>
```

Важно. Действия, выполняемые **json_decode**:

- **JSON массив** декодирует в **PHP массив**.
- **JSON объект** декодирует в **PHP объект / ассоциативный PHP массив** (в зависимости от флага).

Примеры преобразований

При работе с **многомерными массивами** не забывайте – принцип преобразования соответствует основным правилам преобразования строк JSON. Значением массива или объекта может быть:

- число (целое или с плавающей точкой);
- строка (в двойных кавычках);
- логическое значение (true или false);
- другой массив (заклученный в квадратные скобки);
- другой объект (заклученный в фигурные скобки);

- значение null.

Навык работы с массивами и внимательность помогут разобраться со следующими примерами JSON-преобразований.

Кодирование **двумерного индексного** массива.

example_7. JSON-кодирование двумерного индексного массива

```
<?php

// двумерный индексный массив

$array_auto = [

    ['Toyota', 'Toyota Land Cruiser', 'Дизельный', '249'],

    ['BMW', 'BMW 3', 'Дизельный', '190'],

    ['Mitsubishi', 'Mitsubishi Pajero', 'Дизельный', '181']

];

echo "<h2>Двумерный индексный массив</h2>";

echo "<pre>";

var_dump($array_auto);

echo "</pre>";

// преобразуем массив в json представление

$json_auto = json_encode($array_auto);

echo "<h2>JSON-представление двумерного индексного массива</h2>";

echo "<pre>";

var_dump($json_auto);

echo "</pre>";

?>
```

Кодирование **двумерного ассоциативного** массива.

example_8. JSON-кодирование двумерного ассоциативного массива

```
<?php

// двумерный ассоциативный массив

$array_auto = [

    ['brand'=>'Toyota', 'model'=>'Toyota Land Cruiser',
    'engine'=>'Дизельный', 'power'=>'249'],

    ['brand'=>'BMW', 'model'=>'BMW 3', 'engine'=>'Дизельный',
    'power'=>'190'],

    ['brand'=>'Mitsubishi', 'model'=>'Mitsubishi Pajero',
    'engine'=>'Дизельный', 'power'=>'181']

];

echo "<h2>Двумерный ассоциативный массив</h2>";

echo "<pre>";

var_dump($array_auto);

echo "</pre>";

// преобразуем массив в json представление

$json_auto = json_encode($array_auto);

echo "<h2>JSON-представление двумерного ассоциативного
массива</h2>";

echo "<pre>";

var_dump($json_auto);

echo "</pre>";

?>
```

Декодирование JSON-объекта в ассоциативный PHP массив.

example_9. Декодирование JSON объекта в ассоциативный PHP массив

```
<?php

// двумерный ассоциативный массив
```

```

$array_auto = [

    ['brand'=>'Toyota', 'model'=>'Toyota Land Cruiser',
    'engine'=>'Дизельный', 'power'=>'249'],

    ['brand'=>'BMW', 'model'=>'BMW 3', 'engine'=>'Дизельный',
    'power'=>'190'],

    ['brand'=>'Mitsubishi', 'model'=>'Mitsubishi Pajero',
    'engine'=>'Дизельный', 'power'=>'181']

];

// преобразуем массив в json строку

$json_auto = json_encode($array_auto);

echo "<h2>JSON-представление двумерного ассоциативного
массива</h2>";

echo "<pre>";

var_dump($json_auto);

echo "</pre>";

// преобразуем строку json в ассоциативный массив

$new_array = json_decode($json_auto, JSON_OBJECT_AS_ARRAY);

echo "<h2>Двумерный ассоциативный массив</h2>";

echo "<pre>";

var_dump($new_array);

echo "</pre>";

```

?>

Обработка ошибок

Когда **json_encode** или **json_decode** не удастся проанализировать предоставленную строку, они вернут **false**. Сам PHP не вызывает никаких ошибок или предупреждений, когда это происходит.

Пользователь должен использовать функции **json_last_error()** и **json_last_error_msg()**, чтобы проверить, произошла ли ошибка и действовать соответственно поставленной задаче (показать сообщение об ошибке, отладить ее, и так далее).

json_last_error

Функция **json_last_error ()** — возвращает последнюю ошибку.

Описание

Возвращает последнюю ошибку (если она есть), произошедшую во время последнего кодирования/декодирования JSON.

Параметры

У функции нет параметров.

Возвращаемые значения

Возвращает целочисленное значение, которое, может быть, одной из следующих констант*:

Константа	Значение
JSON_ERROR_NONE	Ошибок нет
JSON_ERROR_DEPTH	Достигнута максимальная глубина стека
JSON_ERROR_STATE_MISMATCH	Неверный или некорректный JSON
JSON_ERROR_CTRL_CHAR	Ошибка управляющего символа, возможно неверная кодировка
JSON_ERROR_SYNTAX	Синтаксическая ошибка
JSON_ERROR_UTF8	Некорректные символы UTF-8, возможно неверная кодировка

* - полный список констант можно посмотреть в открытых источниках

json_last_error_msg

Функция **json_last_error_msg ()** — возвращает **строку с сообщением** об ошибке последнего вызова json_encode или json_decode.

Описание

Возвращает текстовое описание последней ошибки, произошедшей при выполнении json_encode или json_decode.

Параметры

У функции нет параметров.

Возвращаемые значения

Возвращает сообщение об ошибке в случае успешного выполнения или "No error", если ошибки не произошло.

В следующем демонстрационном примере показана **ошибка** при работе с JSON, вызванная неспособностью кодировать строку из-за использования неподдерживаемой кодировки.

В случае успешного кодирования происходит отправка на сервер JSON строки через GET-параметр ссылки.

example_10/index.php. Обработка ошибки кодирования

```
<?php

// переменная для отправки на условный сервер

$val = "Привет, Мир!";

// перекодировем переменную в utf-32

// $val = iconv("", "UTF-32", $val);

// инициализация ассоциативного массива

$arr = ["word" => $val];

// кодируем массив в json объект

$json = json_encode($arr);
```

```

// проверяем успешность кодирования

if (json_last_error() != JSON_ERROR_NONE) {

    printf ("

        JSON string encoding error: %s (error code: %s)",

        json_last_error_msg(),

        json_last_error()

    );

} else

    echo "<a href='target.php?data=" . $json .

        "'>Вперед</a>";

?>

```

Закомментируйте строку преобразующую переменную к кодировке UTF-32 и получите работоспособный код.

```

// перекодируем переменную в utf-32

// $val = iconv("", "UTF-32", $val);

```

В демонстрационном примере **example_11** происходит обработка и вывод принятых данных.

example_11/target.php. Принятие JSON строки в качестве GET-параметра

```

<?php

if(isset($_GET['data'])) {

    // получаем get-параметр строки запроса

    $get = $_GET['data'];

    // параметром является json строка

    // декодируем строку в ассоциативный массив

    $arr = json_decode($get, true);

    echo "<h1>" . $arr['word'] . "</h1>";
}

```

```
}  
?>
```

Итоги

JSON (JavaScript Object Notation) — текстовый формат обмена данными, основанный на JavaScript, но может использоваться в любом языке программирования. Легко читается человеком и машиной. Часто используется в REST API (чаще, чем XML из-за лучшей читабельности и меньшего веса).

Корректные значения JSON:

- **JSON объект** — неупорядоченное множество пар "ключ:значение", заключённое в фигурные скобки { };
- **JSON массив** — упорядоченный набор значений, разделённых запятыми. Находится внутри квадратных скобок [];
- **число** (целое или вещественное);
- литералы **true** (логическое значение "истина"), **false** (логическое значение "ложь") и **null**;
- **строка**.